🐍 Python Error Handling

🟦 **Core idea:** Error handling helps your program respond properly instead of crashing unexpectedly.

---

1. ❗ What is an Error?

An **error/exception** occurs when Python cannot normally perform an operation.

```
number = 10 / 0
```

Python raises:

```
ZeroDivisionError
```

Without error handling:

```
Wrong operation
      ↓
Exception occurs
      ↓
Program may stop
```

Error handling means controlling these situations so the application can respond properly instead of unexpectedly crashing.

---

2. 🛡️ `try` and `except`

Basic structure:

```
try:
    risky_code

except SomeError:
    handle_error
```

Example:

```
try:
    number = int(input("Enter a number: "))

except ValueError:
    print("Please enter a valid number.")
```

If the user enters:

```
25
```

the `try` succeeds.

If the user enters:

```
hello
```

`ValueError` occurs and `except` handles it.

🔄 Flow

```
try
 ↓
Success → continue normally

Error
 ↓
matching except
 ↓
handle the error
```

---

3. 🔍 What Happens When an Error Occurs Inside `try`?

```
try:
    print("A")
    number = int("hello")
    print("B")

except ValueError:
    print("C")

print("D")
```

Output:

```
A
C
D
```

Why?

```
A → runs
int("hello") → ValueError
B → skipped
except → C runs
D → runs
```

⚠️ **Important:** Once an exception occurs, Python skips the remaining statements inside that `try` block and searches for a matching `except`.

---

4. 📚 Common Exception Types

Exception	Common cause	Example
`ValueError`	Correct type but invalid value/format	`int("hello")`
`TypeError`	Incompatible data types/operation	`"5" + 10`
`ZeroDivisionError`	Division by zero	`10 / 0`
`IndexError`	Invalid list/sequence index	`[1,2][10]`
`KeyError`	Missing dictionary key	`data["age"]`
`NameError`	Variable doesn't exist	`print(username)`
`FileNotFoundError`	Requested file doesn't exist	`open("x.txt")`
`AttributeError`	Object doesn't have requested attribute	`student.name` when absent
`ImportError` / **`ModuleNotFoundError`**	Import problem	`import unknown_module`

---

5. 🎯 Specific vs General `except`

Broad:

```
try:
    number = int(input())

except:
    print("Something went wrong.")
```

This catches almost everything and is generally **not preferred in professional code**.

Specific:

```
try:
    number = int(input())

except ValueError:
    print("Please enter a valid number.")
```

Specific exceptions are better because you know **what problem you are handling**.

✅ Rule

Catch the most specific exception you reasonably expect.

---

6. 🧩 Multiple `except` Blocks

Different errors can have different responses.

```
try:
    number = int(input("Enter number: "))
    result = 100 / number

except ValueError:
    print("Please enter a valid number.")

except ZeroDivisionError:
    print("You cannot divide by zero.")
```

Flow:

```
Invalid input → ValueError
Zero → ZeroDivisionError
Valid number → calculation
```

---

7. 🔗 Catching Multiple Exceptions Together

If different exceptions need the **same handling**:

```
try:
    number = int(input())
    result = 10 / number

except (ValueError, ZeroDivisionError):
    print("Invalid operation.")
```

Use separate `except` blocks when different errors need different messages or actions.

---

8. 📌 Order of `except` Blocks

Specific exceptions should come before general exceptions.

❌ Poor order:

```
try:
    number = int(input())

except Exception:
    print("General error")
```

```
except ValueError:
    print("Invalid number")
```

`Exception` is broad and can already catch `ValueError`.

✅ Better:

```
try:
    number = int(input())

except ValueError:
    print("Invalid number")

except Exception:
    print("Unexpected error")
```

Rule:

```
Specific
   ↓
More general
   ↓
Exception
```

---

9. 📜 `Exception as e`

Use `as e` when you need information about the exception.

```
try:
    number = int("hello")

except ValueError as e:
    print(e)
```

Possible output:

```
invalid literal for int() with base 10: 'hello'
```

🛠️ Development/debugging

```
except ValueError as e:
    print(e)
```

👤 User-facing application

```
except ValueError:
    print("Please enter a valid number.")
```

⚠️ Don't normally expose technical exception messages directly to end users.

---

10. ✅ `else`

`else` executes **only when no exception occurs** in `try`.

```
try:
    number = int(input("Enter number: "))

except ValueError:
    print("Invalid number.")

else:
    print(f"You entered {number}.")
```

Flow:

```
try
├── error → except
└── no error → else
```

Useful when you want to separate risky operations from code that should execute after successful validation.

---

11. 🧹 `finally`

`finally` executes whether an exception occurs or not.

```
try:
    number = int(input())

except ValueError:
    print("Invalid input.")

finally:
    print("Program finished.")
```

Both valid and invalid input eventually execute:

```
Program finished.
```

Main purpose

`finally` is especially useful for **cleanup**, such as closing resources.

```
try      → attempt operation
except   → handle failure
else     → successful case
finally  → always perform cleanup
```

---

12. 🧱 Complete Structure

```
try:
    risky_code

except SpecificError:
    handle_error

except AnotherError:
    handle_another_error

else:
    run_if_successful

finally:
    always_run
```

You don't always need all four.

---

13. 🚨 `raise`

`raise` allows **you to intentionally generate an exception**.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative.")
```

Python itself allows:

```
age = -5
```

because `-5` is a valid integer.

But your **application rule** says age cannot be negative.

So:

```
raise ValueError(...)
```

communicates:

This value violates my application's rules.

---

14. ⚖️ Python Error vs Application Validation

Python error

Python cannot perform the operation:

```
int("hello")
```

→ `ValueError`

Application validation

Python can accept the value, but your application rejects it:

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative.")
```

Remember:

```
Python rules
+
Application rules
=
Complete validation
```

---

15. 🧠 `raise` Inside a Function

```
def calculate_age(age):

    if age < 0:
        raise ValueError("Age cannot be negative.")

    return age
```

Valid:

```
calculate_age(21)
```

Invalid:

```
calculate_age(-5)
```

The function raises the exception instead of returning an invalid result.

---

16. 🔁 `return` vs `raise`

`return`	`raise`
Gives a normal result	Signals a problem
Function completed normally	Function cannot continue normally
`return result`	`raise ValueError(...)`

Example:

```
def square(number):
    if number < 0:
        raise ValueError("Negative number not allowed.")

    return number * number
```

Think:

```
return → "Here is the result."

raise → "Something is wrong."
```

---

17. 📍 Where Should We Handle the Exception?

A function can allow an exception to reach its caller.

```
def divide(a, b):
    return a / b
```

Then:

```
try:
    result = divide(10, 0)

except ZeroDivisionError:
    print("Cannot divide by zero.")
```

This can be cleaner because:

```
Function
→ performs its responsibility

Caller
→ decides how to handle failure
```

Don't automatically put `try/except` inside every function.

---

18. 😲 `pass` in `except`

```
try:
    number = int("hello")

except ValueError:
    pass
```

`pass` means:

Do nothing.

Although syntactically valid, blindly ignoring exceptions is usually poor practice because it can hide bugs.

Avoid:

```
except Exception:
    pass
```

unless there is a very specific reason.

---

19. 🔄 `while` + Error Handling

For user input, you often want to ask again instead of terminating.

```
def get_number():

    while True:
        try:
            number = int(input("Enter a number: "))
            return number

        except ValueError:
            print("Invalid input. Try again.")
```

Flow:

```
Ask
 ↓
Valid?
 ├── Yes → return
 └── No → error message → ask again
```

This is a very useful pattern for CLI applications.

---

20. ✅ Validation + Error Handling

```
def get_positive_number():

    while True:
        try:
            number = int(input("Enter a positive number: "))

            if number <= 0:
                print("Number must be greater than zero.")
                continue

            return number

        except ValueError:
            print("Please enter a valid integer.")
```

Here:

- `ValueError` handles invalid conversion.
- `if` handles application validation.
- `continue` restarts the loop.
- `return` ends the function with valid data.

---

21. ✂️ Why `try` Should Be Small

❌ Avoid unnecessarily huge blocks:

```
try:
    # 50 lines of code
except Exception:
    print("Error")
```

If something fails, it becomes difficult to determine where and why.

✅ Better:

```
name = input("Enter name: ")

try:
    age = int(input("Enter age: "))

except ValueError:
    print("Age must be a number.")
```

💡 Professional rule

Keep the `try` block as small as reasonably possible.

Only protect the operation that can actually raise the expected exception.

---

22. 📝 Real Project Example — To-Do App

Suppose:

```
tasks = [
    "Learn Python",
    "Learn OOP",
    "Build project"
]
```

A user selects a task to delete.

```
while True:
    try:
        task_number = int(input("Enter task number: "))

        if task_number < 1 or task_number > len(tasks):
            print("Task does not exist.")
            continue

        deleted_task = tasks.pop(task_number - 1)

        print(f"{deleted_task} deleted successfully.")
        break

    except ValueError:
        print("Please enter a valid number.")
```

Possible problems:

```
hello → ValueError
100   → application validation failure
1     → successful deletion
```

This is how error handling becomes part of real application logic.

---

23. 🎭 Expected vs Unexpected Errors

✅ Expected

A user entering:

```
hello
```

when a number is required is an expected situation.

Handle it clearly:

```
except ValueError:
    print("Please enter a valid number.")
```

⚠️ Unexpected

Something goes wrong that your normal logic did not anticipate.

A general handler can sometimes be used at an appropriate application boundary:

```
except Exception as e:
    print(f"Unexpected error: {e}")
```

But don't use `except Exception` everywhere.

---

24. 🏆 Professional Error-Handling Principles

Principle	Good practice
Catch specific errors	Prefer `ValueError` over bare `except`
Keep `try` small	Protect only risky operations
Don't hide errors	Avoid unnecessary `pass`
Validate input	Combine exceptions with normal conditions
Use `raise` when appropriate	Signal invalid application state
Use friendly messages	Don't expose technical details unnecessarily
Separate responsibilities	Function performs work; caller can handle failure
Use `finally` for cleanup	Especially resources
Use general `Exception` carefully	Good at appropriate boundaries, not everywhere
Reuse validation logic	Build helper functions

---

25. 🚀 Error Handling in Your Future Projects

📝 To-Do App

```
User input
   ↓
ValueError
   ↓
Validation
   ↓
Safe operation
```

📁 File Organizer

Possible exceptions:

```
FileNotFoundError
PermissionError
FileExistsError
IsADirectoryError
NotADirectoryError
```

🤖 API/AI Application

Possible problems:

```
Connection errors
Timeouts
Invalid responses
JSON decoding errors
Authentication errors
```

📄 File Handling

Example:

```
try:
    with open("data.txt", "r") as file:
        data = file.read()

except FileNotFoundError:
    print("File does not exist.")
```

This connects directly to the next major Python topic: **File Handling**.

---

26. 🎤 Interview Quick Answers

What is exception handling?

Exception handling is a mechanism for detecting and handling runtime exceptions so that a program can respond appropriately instead of terminating unexpectedly.

Difference between error and exception?

An exception is a runtime problem that Python can potentially handle using mechanisms such as `try/except`. In Python discussions, "error" is often used broadly for both exceptions and other programming problems.

What is `try`?

It contains code that may raise an exception.

What is `except`?

It handles a matching exception.

What is `else`?

It runs when the `try` block completes without an exception.

What is `finally`?

It runs regardless of whether an exception occurs and is commonly used for cleanup.

What is `raise`?

It manually raises an exception.

Why avoid bare `except`?

Because it can catch unintended exceptions and make debugging difficult.

What is `Exception`?

`Exception` is a broad base class for most normal, non-system-exiting exceptions.

Why should specific exceptions come first?

Because a broad exception handler such as `Exception` can also catch more specific exceptions.

---

27. 🎯 Most Expected Interview Questions for Your AI Python Developer Goal

Foundation

1. What is exception handling in Python?
2. Explain `try`, `except`, `else`, and `finally`.

3. What is the difference between an exception and a syntax error?
4. What happens when an exception occurs inside a `try` block?
5. Can one `try` have multiple `except` blocks?
6. What is the difference between `except:` and `except Exception:`?
7. Why should you avoid bare `except`?
8. How do you catch multiple exceptions?
9. What does `as e` do in `except Exception as e`?
10. What is the purpose of `finally`?

Practical/Professional

11. When should you use `raise`?
12. What is the difference between `raise` and `return`?
13. Where should exception handling occur: inside a function or at its caller?
14. Why should a `try` block generally be kept small?
15. Why is `except Exception: pass` dangerous?
16. How would you validate user input safely?
17. How would you handle a missing file?
18. How would you handle invalid JSON/API data?
19. How would you design reusable input-validation functions?
20. How would you distinguish expected user errors from unexpected application failures?

AI/API Project-Oriented

21. How would you handle an API request failure?
22. How would you handle an API timeout?
23. How would you handle malformed JSON returned by an API?
24. How would you prevent one failed API request from crashing your application?
25. Where would you place error handling in a multi-layer application?
26. Why should user-facing applications not expose raw exception messages?
27. How would you log unexpected exceptions while showing a friendly message to the user?
28. How would you combine validation and exception handling in an AI application?
29. What exceptions might occur when reading uploaded files?
30. How would you design robust error handling for an AI-powered application?

🧠 One-Line Memory Formula

```
try     → Try risky code
except  → Handle failure
else    → Handle success
finally → Always clean up
raise   → Create/report a problem
```